

# UC, Categorically: Rigorous Diagrammatic Proofs

Pooya Farshim<sup>\*</sup>, Martti Karvonen<sup>†</sup>, Andre Knispe<sup>‡</sup>, Markulf Kohlweiss<sup>§¶</sup>, Philip Wadler<sup>§</sup>

Input Output, <sup>\*</sup>Switzerland, <sup>‡</sup>Germany, <sup>§</sup>UK,

<sup>†</sup>University College London, UK, <sup>¶</sup>University of Edinburgh, UK

**Abstract**—Category theory is a mathematical theory of composition, widely used in logic, computing, and physics. Here we apply it to give a theory of secure composition. In particular, we provide a categorical treatment of Canetti’s Universal Composability (UC) framework for systems with a static number of parties and sessions, often termed UC for static systems (UC4SS), yielding four benefits.

First, we present our results graphically yet retain rigor by applying a standard categorical technique known as *string diagrams*. In particular, our formulation of the composition theorem can be graphically verified with a short sequence of diagrams, while remaining translatable to equations and amenable to formal verification.

Second, categories let us generalize so that our results do not apply only to interactive Turing machines but also to other forms of computation, such as quantum or domain-specific languages.

Third, categories help us to drop some unnecessary restrictions of UC (e.g., our adversary can be a computational network rather than a single Turing machine); we prove equivalence between our variant and the usual UC4SS, showing no expressiveness is lost. Finally, categories lead us to fix some technical errors which have gone undetected in the eight years since UC4SS was first published.

**Index Terms**—Category Theory, Universal Composition, Symmetric Monoidal Category, String Diagram.

## I. INTRODUCTION

Despite—or perhaps because of—being hugely influential, Universal Composability (UC) [1], [2] has divided, rather than unified, industrial and academic cryptographers as well as game-based and simulation-based provable-security experts. Those investigating simulation-based composable security are further divided into (at least) the Constructive Cryptographers [3] and the IITM [4] camp. Historians of science will point out that this is nothing new and only to be expected in a relatively young discipline. What then can be learned from unification in other areas, namely, mathematics?

Category Theory is the mathematics of composition. It is used as a unifying language in much of mathematics, and it is widely used in logic, computing, and physics, not least to reveal connections between the three [5]. It is perhaps surprising that despite its unifying successes in Mathematics and Computer Science, this mathematics of composition has not yet been comprehensively applied to secure composition.

The purpose of this paper is to express (a minor variant of) UC4SS [2, Section 2], henceforward often called just UC, and its composition theorem in categorical terms. While this restriction is done for technical simplicity, it should be noted that simple UC (also known as UC for static systems) is already fairly expressive and can model, e.g., multi-party computation with an apriori known set of participants [6] or any protocol

that uses a static number of idealized building blocks. Indeed, EasyUC [7] works essentially in this setting. Also note that UC4SS is all you need when considering concrete security and simplifies the treatment of global functionalities [8], though the latter are not the focus of this work.

Our approach offers several benefits:

*First*, the use of categories allows us to import the machinery of *string diagrams*, an intuitive yet rigorous diagrammatic method for reasoning developed extensively by category theorists. As a result, our proofs of the Composition Theorems III.8 and III.9 are easier to follow than textual presentations, yet translatable into equations and amenable to computer formalization.

*Second*, categories are very helpful for generalization. To quote Mac Lane [9, IX.6], one of the co-founders of category theory,

The point of these observations is not the reduction of the familiar to the unfamiliar [...] but the extension of the familiar to cover many more cases.

Indeed, it is readily apparent that our proof holds for models of computation other than UC4SS’s networks of interactive Turing machines (ITMs). Specifically, our proof gives a template for developing UC-like theories where ITMs are replaced by, e.g., quantum systems (giving a variant of quantum UC [10]) or by code in some suitable domain-specific language (DSL) for UC [7], [11]–[13]. In particular, one does not need to reprove the composition theorem when developing such further variants: essentially, to invoke the general result one just needs to verify that one’s notion of networks of “interactive computational systems” admits a notion of computational indistinguishability satisfying Lemma IV.1.

*Third*, categories tend to “point you in the right direction”:

- There is no need for machines to have names (identities) in the context of UC4SS: instead, it is convenient to name the connections between machines.
- There is no need to restrict the environment and the adversary to be a single machine; they can be allowed to consist of several machines, just like protocols. As a corollary, results requiring absorbing some machine into the environment/adversary are immediate as they do not require an additional step of replacing the resulting network by a single equivalent machine.

Despite these changes, we show in Section IV-D that our version can be translated to UC4SS (and vice versa) as-is, letting us deduce [2, Theorem 3] from our results.

*Finally*, we fix some technical oversights (in Section IV-D) around the composition theorem for UC4SS which are imme-

diately suggested by the basic categorical principle that two things should only be composed sequentially if their types match suitably, i.e., if the codomain of one is the domain of the other. To our knowledge, these oversights have gone unnoticed for eight years, since the section on UC4SS was added in [2, Section 2], and might have gone unnoticed longer without the categorical perspective.

We envision that further benefits will materialize once the program has been carried out in further detail:

- Our work paves the way for developing a variant of UC in terms of a domain-specific language (DSL), thus making rigorous the widespread practice of working in UC while using (pseudo)code to express computational behaviors (indeed, we are not aware of *any* examples of someone writing down an ITM when using UC).
- We intend to provide a similar analysis for several other frameworks for composability, such as Abstract/Constructive Cryptography [3], [14] (and specifically random systems [15] as the underlying computational model), the IITM model [16], state-separating proofs [17], reactive simulatability [18]–[20], or other approaches [7], [12], [13], [21]–[23]. This will result in a single, categorically expressed, composition theorem unifying specific composition theorems. In particular, the categorical axiomatization could be used as a guideline when modifying existing frameworks or developing altogether novel ones: as long as the axioms are satisfied, composition will come for free.
- Perhaps more importantly, once some frameworks have been expressed in categorical terms, it will become easier to rigorously compare and translate between them, resulting in security-preserving translations akin to [24] between different frameworks.

*Structure of the paper.* In Section II we briefly introduce categories and string diagrams. The main novelties are introduced in Section III where we express UC categorically, with the main results being: Theorem III.8 guaranteeing that security is closed under sequential and parallel composition, and Theorem III.9 giving our categorical version of the composition theorem for UC4SS. We then specialize to UC and give a translation between our setting and UC4SS [2, Section 2] in Section IV. We conclude with some further questions in Section V.

#### A. Related works

This paper fits into a larger program, initiated in [25], [26], of using category theory for questions of composable security in cryptography. Our main text is self-contained (at the cost of brushing over some categorical intricacies) and expresses UC directly in its own right. However, in Appendix A, we show that our composition theorems (Theorems III.8 and III.9) can be deduced from (a minor variant of) the general composition theorem presented in [26, Theorem 4.6]. When doing so we assume more categorical knowledge, and relegate some of the explanations to [26]. Proving the composition theorem directly is easier and hence done in the main text, while expressing

it as an instance of the more general result is helpful for the future goal of unification. Our goal here is not to be maximally general, but to give a template for building “UC-like” theories with more ease than given by the general framework of [26].

The use of informal diagrams as a heuristic device is common in cryptography (see, e.g., [17], [27]–[29]) and recasting them as string diagrams promotes them into a rigorous tool without losing their intuitive nature. We note that string diagrams have already been used in quantum cryptography when analyzing particular protocols (see e.g., [30]–[35]), whereas here we use them in the foundations of composable security. Facilitating and popularizing the use of string diagrams in cryptography is one of our long-term goals.

## II. A CRASH COURSE ON CATEGORY THEORY

This section introduces the category theory needed for our purposes. String diagrams are surveyed in [36], [37]. A working cryptographer might find texts such as [38]–[40] easier to learn from, as they start with some applications in mind and introduce string diagrams concurrently with the material. General references for category theory include [41], [42].

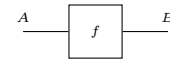
### A. Categories

In mathematics and computing we often study objects of a given kind together with maps between those objects, such as:

- sets and functions between sets;
- groups and group homomorphisms;
- finite-dimensional real vector spaces (with a chosen basis) where maps from a space of dimension  $m$  to a space of dimension  $n$  are given by  $m \times n$  matrices;
- types in some type system, where maps from type  $S$  to type  $T$  consist of a well-typed term  $t$  of type  $T$  with a free variable  $x$  of type  $S$  (in shorthand,  $x : S \vdash t : T$ ).

Category theory provides a systematic language to capture this kind of situation. A *category*  $\mathbf{C}$  consists of *objects* and *maps* between them. We denote objects  $A, B, C, \dots$  and maps, also called *morphisms*, by  $f, g, h, \dots$ . Every morphism has a *source* (also called the *domain*) and a *target* (also called the *codomain*), and we write  $f : A \rightarrow B$  to mean that  $f$  is a morphism (in  $\mathbf{C}$ ) from  $A$  to  $B$ .

A nice feature of categories is that they admit a pictorial syntax, known as *string diagrams* that is both intuitive and rigorous. We introduce string diagrams in tandem with our discussion of categories. In a string diagram, a morphism  $f : A \rightarrow B$  is depicted by



We will often omit the labels for the wires when there is no risk of confusion.

In a category, it’s crucial that morphisms can be composed whenever it makes sense to do so. For our four examples above

- In the category **Set** of sets and functions, if  $f : A \rightarrow B$  and  $g : B \rightarrow C$  their composition is given by  $g \circ f : A \rightarrow C$ .
- In the category **Grp** of groups and group homomorphisms composition is defined similarly.

- In the category **Mat** of vector spaces and matrices, composition corresponds to matrix multiplication.
- In the category **Trm** of types and terms, the composition of  $x: S \vdash t: T$  with  $y: T \vdash s: U$  is given by substituting  $t$  for  $y$  in  $s$ , giving  $x: S \vdash s[t/y]$ .

In general, in a category **C**, there has to be a sequential composition operation  $\circ$ , which, when given  $f: A \rightarrow B$  and  $g: B \rightarrow C$ , returns  $g \circ f: A \rightarrow C$ , which in string diagrams is expressed by

$$\begin{array}{c} A \quad \boxed{g \circ f} \quad C \\ \hline \end{array} := \begin{array}{c} A \quad \boxed{f} \quad B \quad \boxed{g} \quad C \\ \hline \end{array}$$

To match the diagrammatic order, the notation  $f;g$  is sometimes used instead of  $g \circ f$ .

Moreover, for every object  $A$  there has to be an identity morphism  $\text{id}_A: A \rightarrow A$  on it, drawn as

$$\begin{array}{c} A \quad \text{---} \quad A \\ \hline \end{array}$$

What makes  $\text{id}_A$  and  $\text{id}_B$  earn their names is the fact that they're required to satisfy  $f \circ \text{id}_A = f = \text{id}_B \circ f$  for any  $f: A \rightarrow B$ . One benefit of string diagrams is how they make the required axioms pictorially intuitive, so this would be expressed by

$$\begin{array}{c} A \quad A \quad \boxed{f} \quad B \\ \hline \end{array} = \begin{array}{c} A \quad \boxed{f} \quad B \\ \hline \end{array} = \begin{array}{c} A \quad \boxed{f} \quad B \quad B \\ \hline \end{array}$$

More informally, this amounts to saying that the length of a wire carries no information.

Moreover, we require composition to be associative, so that whenever we have three morphisms  $f: A \rightarrow B, g: B \rightarrow C$  and  $h: C \rightarrow D$  that could be composed in sequence, we don't need to specify which pairs we compose first, i.e.,  $h \circ (g \circ f) = (h \circ g) \circ f$ . This means that we can omit the brackets and simply write  $h \circ g \circ f$ , which justifies the drawing

$$\begin{array}{c} A \quad \boxed{f} \quad \boxed{g} \quad \boxed{h} \quad D \\ \hline \end{array}$$

This concludes the definition of a category: a class of objects and morphisms between them where morphisms compose, there is an identity morphism for each object, the identity morphisms are identities under composition, and composition is associative.

The structure of a category is already sufficiently rich to express many properties of interest. For example, one can define a morphism  $f: A \rightarrow B$  to be an *isomorphism* if there exists a morphism  $g: B \rightarrow A$  such that  $g \circ f = \text{id}_A$  and  $f \circ g = \text{id}_B$ . This general notion specializes to the correct notion of “sameness” in each context, capturing, e.g., bijections between sets and isomorphisms of groups, graphs, and vector spaces and types.

While many examples of interest arise from sets-with-further-structure and structure-preserving maps, it is important to note that the definition of a category does not require the morphisms to consist of some kind of functions. To see that this is not always the right thinking, it is useful to consider a partially ordered set (poset) consisting of a set  $P$

equipped with a relation  $\leq$  that is reflexive, transitive, and antisymmetric. Any poset  $\langle P, \leq \rangle$  induces a category whose objects are given by elements of  $P$ , and whose morphisms are defined by setting there to be exactly one morphisms  $x \rightarrow y$  whenever  $x \leq y$  and no morphisms  $x \rightarrow y$  otherwise. In fact, we'll see another example of a category where the intuition of morphisms as functions is not correct: in Section IV-A we'll encounter a category whose morphisms are certain networks of interactive Turing machines.

## B. Monoidal and symmetric monoidal categories

In many categories of interest, there is also a way of composing objects and morphisms *in parallel* and not just sequentially. For example,

- Recall that if  $A$  and  $B$  are sets then the cartesian product  $A \times B = \{\langle x, y \rangle \mid x \in A, y \in B\}$  is the set of all pairs of elements with the first element in  $A$  and the second in  $B$ . Given two functions  $f: A \rightarrow B$  and  $g: C \rightarrow D$ , there is a function  $f \times g: A \times C \rightarrow B \times D$  which sends  $\langle x, y \rangle \in A \times C$  to  $\langle f(x), g(y) \rangle \in B \times D$ .
- Given two groups  $G$  and  $H$ , the product  $G \times H$  is obtained by taking the cartesian product of the underlying sets, and doing multiplication pointwise. With this definition, the product  $f \times g$  of group homomorphisms  $f$  and  $g$  is again a group homomorphism.
- Given two vector spaces  $A$  and  $B$  with bases  $a_1, \dots, a_m$  and  $b_1, \dots, b_n$ , their tensor product  $A \otimes B$  is an  $m \times n$ -dimensional vector space that has a basis built out of pairs  $(a_i, b_j)$ . Now, given  $n \times m$ -matrix  $f$  and a  $p \times q$ -matrix  $g$ , their *Kronecker product*  $f \otimes g$  is the  $nq \times mp$ -matrix essentially obtained by replacing the  $(i, j)$ th entry  $f_{i,j}$  of  $f$  with the matrix  $f_{i,j}g$ .
- If a type system has product types, one can use terms  $x: S \vdash s: T$  and  $y: U \vdash v: V$  to obtain a term  $z: S \times U \vdash \langle s[\text{fst}(z)/x], t[\text{snd}(z)/y] \rangle: T \times V$ , where  $\langle s, t \rangle$  builds a pair and  $\text{fst}$  and  $\text{snd}$  extract the first and second components of a pair.

Monoidal categories capture this kind of situation. Roughly speaking, a monoidal category is a category equipped with a parallel composition operation that is associative and has a unit. The parallel composition operation is written using the tensor product symbol  $\otimes$ . Given two morphisms  $f: A \rightarrow B$  and  $g: C \rightarrow D$  in **C**, their parallel composite is a morphism  $f \otimes g: A \otimes C \rightarrow B \otimes D$ . In pictures this is denoted by juxtaposition:

$$\begin{array}{c} A \quad \boxed{f \otimes g} \quad B \\ C \quad \quad \quad D \end{array} := \begin{array}{c} A \quad \boxed{f} \quad B \\ C \quad \boxed{g} \quad D \end{array}$$

This parallel composition operation must satisfy  $\text{id}_{A \otimes B} = \text{id}_A \otimes \text{id}_B$  which pictorially amounts to

$$\begin{array}{c} A \otimes B \quad A \otimes B \\ \hline \end{array} = \begin{array}{c} A \quad \text{---} \quad A \\ B \quad \text{---} \quad B \end{array}$$

This lets us replace a wire of type  $A \otimes B$  with two wires (one for  $A$  and one for  $B$ ) whenever it is convenient to

do so. Morphisms between products are not required to be products of morphisms. For example, a function  $(x, y) \mapsto (f(x, y), g(x, y))$  is a product only if  $f$  depends only on  $x$  and  $g$  depends only on  $y$ .

An arbitrary box can have multiple input and output wires. In pictures, a morphism

$$f: A_1 \otimes \cdots \otimes A_n \rightarrow B_1 \otimes \cdots \otimes B_m$$

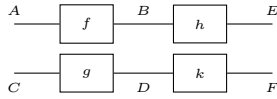
is drawn as



We also require the interchange law, which states that

$$(h \otimes k) \circ (f \otimes g) = (h \circ f) \otimes (k \circ g)$$

for all  $f: A \rightarrow B, g: C \rightarrow D, h: B \rightarrow E$  and  $k: D \rightarrow F$ . The interchange law guarantees that the string diagram



is unambiguous.

In a *strict* monoidal category, parallel composition is *unital*. This means that there is a special object  $I$ , called *the tensor unit*, satisfying

$$A \otimes I = I \otimes A = A \quad \text{and} \quad \text{id}_I \otimes f = f = f \otimes \text{id}_I$$

on objects and morphisms, respectively. Further, parallel composition is *associative*, meaning

$$A \otimes (B \otimes C) = (A \otimes B) \otimes C \quad \text{and} \quad f \otimes (g \otimes h) = (f \otimes g) \otimes h.$$

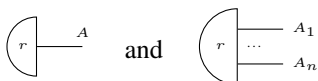
This results in the notion of a *strict monoidal category*.

Unfortunately, strict monoidal categories are too strict—they rule out most examples of interest. For instance, the cartesian product of sets is not strictly associative. Rather, it is associative up to isomorphism,  $((a, b), c) \mapsto (a, (b, c))$ . Loosely speaking, a *monoidal category* is a category with parallel composition  $\otimes$  that is associative and unital up to isomorphism. More formally, a monoidal category is a category equipped with a parallel composition operation and associativity and unitality isomorphisms which must satisfy some reasonable equations. We won't delve into these equations in painful detail, see [41, Section 7.8] for a precise definition.

The tensor unit  $I$  is denoted by the empty picture (as juxtaposing any picture with the empty one results in the same picture). Thus, a morphism  $r: I \rightarrow A$  would be denoted by a box with no inputs

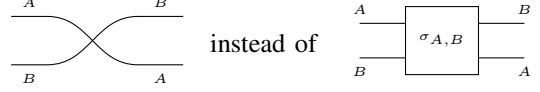


Borrowing terminology from quantum circuits, morphisms from  $I$  are usually called *states*, and we will draw states  $r: I \rightarrow A$  and  $r: I \rightarrow A_1 \otimes \cdots \otimes A_n$  respectively as

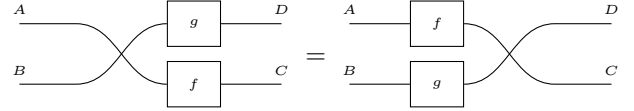


In **Set** these correspond to elements of a set, and in **Mat** these correspond to vectors.

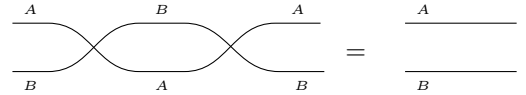
Now, a *symmetric* monoidal category (SMC) is, roughly speaking, a monoidal category where  $\otimes$  is suitably commutative. Slightly more formally, an SMC comes with specified *swap* or *symmetry* isomorphisms  $\sigma_{A,B}: A \otimes B \rightarrow B \otimes A$  for all objects  $A, B$ , satisfying some further equations. These further equations are more easily explained in string diagrams. To start, we draw  $\sigma_{A,B}: A \otimes B \rightarrow B \otimes A$  as



Given this notation, we then require



or equivalently,  $(g \otimes f) \circ \sigma_{A,B} = \sigma_{C,D} \circ (f \otimes g)$  holds for all  $f: A \rightarrow C, g: B \rightarrow D$ . Intuitively, this states that boxes can be slid along wires. (Formally this property corresponds to the family  $\sigma$  being a *natural transformation*, a concept that shows up often in category theory.) Second, we require that crossing the same wires twice amounts to no change:



or equivalently  $\sigma_{B,A} \circ \sigma_{A,B} = \text{id}_{A \otimes B}$ .

Given an SMC  $\mathbf{C}$ , by a *sub-SMC* of  $\mathbf{C}$  we mean an SMC  $\mathbf{D}$  whose objects and morphisms are subcollections of those of  $\mathbf{C}$ , and whose SMC-structure (sequential and parallel composition, identity morphisms, unit object, symmetries) coincides with that of  $\mathbf{C}$ . Sub-SMCs are closed under arbitrary intersections: as a result, one can always form the smallest sub-SMC containing given collections of morphisms and objects, and we will refer to the end-result as the *sub-SMC generated* by this data.

We will also need mappings between SMCs. The maps we will need in the main text are known as *strict symmetric monoidal functors*: such a functor  $F: \mathbf{C} \rightarrow \mathbf{D}$  sends every object  $A$  of  $\mathbf{C}$  to an object  $F(A)$  of  $\mathbf{D}$  and every morphism  $f: A \rightarrow B$  to a morphism  $F(f): F(A) \rightarrow F(B)$ , preserving sequential and parallel composition, identities, tensor unit and the symmetry. For example, the “forgetful functor”  $\mathbf{Grp} \rightarrow \mathbf{Set}$  sending a group to its underlying set (forgetting the group operation) and a group homomorphism to the underlying function (forgetting that it preserves the group operation) is a strict symmetric monoidal functor.

### C. Building an SMC from generators and equations

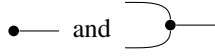
We will now review how to build a symmetric monoidal category from generators and equations, as this will be used in Section IV-A to build a category of networks of interactive

Turing machines in order to model UC4SS. A useful analogy is that of building a group from generators and equations. A more detailed yet introductory account can be found in [37, Chapter 2]. To generate an SMC, one needs

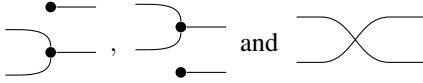
- A set  $\Sigma$  of generating objects, a general object then being an element of the set  $\Sigma^*$  of words in the alphabet  $\Sigma$ .
- For each  $V, W \in \Sigma^*$ , a (possibly empty) set  $\Gamma_{V,W}$  of generating morphisms  $V \rightarrow W$ .

The symmetric monoidal category generated by  $(\Sigma, \Gamma)$  has  $\Sigma^*$  as its set of objects, and its morphisms can be thought of as given by string diagrams built from identity wires, symmetries and generating morphisms from the sets  $\Gamma_{V,W}$  modulo the equations we’ve asserted for SMCs (associativity and unitality of  $\otimes$  and the equations for swap) in the previous section.

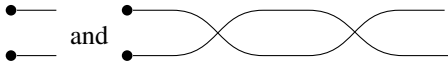
As a warm up for us using this construction in later sections, let us consider an example. We let  $\Sigma$  consist of a single element, so that an element of  $\Sigma^*$  is uniquely specified by its length and hence  $\Sigma^*$  can be identified with  $\mathbb{N}$ . We let  $\Gamma_{0,1}$  and  $\Gamma_{2,1}$  be singletons and draw their elements as



and let  $\Gamma_{i,j}$  be empty for all other choices of  $i, j \in \mathbb{N}$ . Then in the resulting SMC,



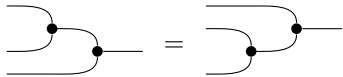
give three distinct morphisms  $2 \rightarrow 2$  whereas



denote the same morphism  $0 \rightarrow 2$  due to equations satisfied by any SMC (swapping twice gives the identity).

Having built an SMC from generators, we now discuss building an SMC from generators and equations. It turns out that there is no real need to allow for equations between objects, as equations between morphisms will be sufficient.<sup>1</sup> In turn, a defining equation, is given by  $f = g$ , where  $f, g$  are two morphisms  $V \rightarrow W$  in the monoidal category generated by  $(\Sigma, \Gamma)$ . In the resulting SMC, morphisms can now be thought of as equivalence classes of string diagrams, modulo the defining equations.

Returning to the simple example, we will now be adding the equations of a commutative monoid. Associativity of the multiplication is captured by

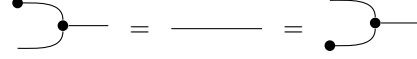


<sup>1</sup>If one did want to identify two words  $A, B \in \Sigma^*$ , one would instead add generating morphisms  $f: A \rightarrow B$  and  $g: B \rightarrow A$  and equations asserting that  $g$  is the inverse of  $f$ . While this doesn’t force  $A$  and  $B$  to be equal in the resulting SMC, it forces them to be isomorphic, which is sufficient for most categorical purposes.

while commutativity is given by



and the unit laws are



One can then build an SMC out of these generators and relations, whose morphisms are then equivalence classes of string diagrams modulo the equations we’ve asserted. In Section IV-A, we will apply this machinery to UC.

### III. UNIVERSAL COMPOSABILITY CATEGORICALLY

#### A. The starting category

The theory we develop starts from an SMC  $\mathcal{C}$ , layers some categorical constructions on top, and culminates in Theorems III.8 and Theorems III.9 giving our composition theorems. As a result, one could instantiate this theory with any of the examples of an SMC discussed so far. However, these are unlikely to be cryptographically meaningful. Closer to cryptography, one could extract SMCs fairly directly from (minor variants of) state-separating proofs [17] or random systems [15], and perhaps with more effort, from elsewhere in cryptography. In general, finding cryptographically meaningful categories off-the-shelf is difficult (cf. [26, Section 9]).

As a result, one might want to build the SMC oneself. This is particularly likely to be useful if one has some notion of an atomic notion of an “interactive computational system” which, when composed results in a *network* of such systems (e.g. two ITMs can be connected to form a network of two ITMs, rather than a single ITM — that the composed network could be simulated by a single ITM is beside the point here) instead of just another similar system (like two functions compose into a function).

We can conveniently build such categories by generators and equations. When describing the resulting category, it is useful to borrow intuitions from UC. In this viewpoint, the generating objects in  $\Sigma$  correspond to basic types of communication channels. As a first approximation, UC has just one basic type, but one could imagine communication channels parametrized by the type of data flowing on them (so that a channel for bit strings is different than a channel for communicating vectors, graphs or elements of a group), or perhaps each channel could have a session type [43], capturing the expected communication pattern (“first I send you a vertex of a graph, then you respond with an integer, giving it’s colour”). Given two words  $V, W$ , one then thinks of the generating set  $\Gamma_{V,W}$  as the set of all atomic/basic computational systems with channels  $V$  on one side of the system and channels  $W$  on the other. For example, in our formalization of UC these will correspond to Turing machines with a given number of input tapes and a given number of subroutine output tapes. One may also add further generators and equations if need be. We will see an instance of this in Section IV-A, where we instantiate our setting with a specific category in order to express UC.

### B. Protocols, hybrids, and the adversary

We now assume that we have chosen our starting category  $\mathbf{C}$ . To help intuition, we think of objects of  $\mathbf{C}$  as representing communication channels (possibly of some particular type) and of morphisms of  $\mathbf{C}$  as representing networks built by connecting interactive computational systems along these channels. We now construct from  $\mathbf{C}$  another SMC  $\mathbf{C}_{\text{adv}}$  and then identify some sub-SMCs of  $\mathbf{C}_{\text{adv}}$  that are important to us. Ultimately, all of these SMCs are only needed to construct the category of our main SMC of interest in Theorem III.8.

As in UC, we will ensure that all protocols have explicit “backdoor” connections for the adversary. This is done by building a category  $\mathbf{C}_{\text{adv}}$  from the category  $\mathbf{C}$ . Objects of this category are pairs  $X = (X_h, X_a)$  of objects of  $\mathbf{C}$ , with the intuition that  $X_h$  is the type of the honest interface and  $X_a$  is the adversarial interface.

A morphism  $(X_h, X_a) \rightarrow (Y_h, Y_a)$  consists of an equivalence class of triples  $(Z, h: X_h \rightarrow Y_h \otimes Z, a: Z \otimes X_a \rightarrow Y_a)$  under a certain equivalence relation, whose motivation we will explain shortly. Intuitively,  $h$  is the honest network with backdoor wires given by  $Z$  and  $a$  is the adversary attacking  $h$  (via  $Z$ ) and “the rest” (via  $X_a$ ). We represent such a triple pictorially by the pair of morphisms

$$\left( \begin{array}{c} x_h \\ \text{---} \end{array} \begin{array}{|c|} \hline h \\ \hline \end{array} \begin{array}{c} y_h \\ \text{---} \\ z \\ \text{---} \end{array}, \begin{array}{c} z \\ \text{---} \\ x_a \\ \text{---} \end{array} \begin{array}{|c|} \hline a \\ \hline \end{array} \begin{array}{c} y_a \\ \text{---} \end{array} \right)$$

Pictorially, the equivalence relation is generated by setting

$$\left( \begin{array}{|c|} \hline h \\ \hline \end{array}, \begin{array}{|c|} \hline c \\ \hline \end{array} \begin{array}{|c|} \hline a \\ \hline \end{array} \right) \sim \left( \begin{array}{|c|} \hline h \\ \hline \end{array} \begin{array}{|c|} \hline c \\ \hline \end{array}, \begin{array}{|c|} \hline a \\ \hline \end{array} \right) \quad (\text{III.1})$$

whenever  $c: Z_1 \rightarrow Z_2$  is some isomorphism built out of swaps and identities. Such maps correspond to permutations of wires. We will use square brackets (whether surrounding a picture or symbols) to denote the corresponding equivalence class.

The interpretation of this is as follows: a morphism consists of the network  $h: X_h \rightarrow Y_h \otimes Z$  of honest machines with adversarial backdoors given by  $Z$ , and of the adversary’s action  $a: Z \otimes X_a \rightarrow Y_a$ : the idea is that in  $\mathbf{C}_{\text{adv}}$  we can separate the honest and adversarial parts of the network, so that we can vary the adversarial part (e.g., by quantifying over it, when defining security). The equivalence relation amounts to the fact that permutations of the backdoor wires may be attributed either to the honest machines or to the adversary without changing the overall behavior.

The composite of  $f = [Z_f, f_h, f_a]: X \rightarrow Y$  with  $g = [Z_g, g_h, g_a]: Y \rightarrow W$  is given pictorially as

$$\left[ \begin{array}{c} x_h \\ \text{---} \end{array} \begin{array}{|c|} \hline f_h \\ \hline \end{array} \begin{array}{|c|} \hline g_h \\ \hline \end{array} \begin{array}{c} w_h \\ \text{---} \\ z_g \\ \text{---} \\ z_f \\ \text{---} \end{array}, \begin{array}{c} z_g \\ \text{---} \\ z_f \\ \text{---} \\ x_a \\ \text{---} \end{array} \begin{array}{|c|} \hline f_a \\ \hline \end{array} \begin{array}{|c|} \hline g_a \\ \hline \end{array} \begin{array}{c} w_a \\ \text{---} \end{array} \right]$$

and in symbols as  $[Z_g \otimes Z_f, (g_h \otimes \text{id}) \circ f_h, g_a \circ (\text{id} \otimes f_a)]$ . In words, the sequential composition is performed by simply composing the honest and adversarial parts.

This category also inherits a symmetric monoidal structure from that of  $\mathbf{C}$ . The monoidal product on objects is done

pointwise so that  $(X_h, X_a) \otimes (V_h, V_a) = (X_h \otimes V_h, X_a \otimes V_a)$ , while the product of morphisms is obtained using both the monoidal product and symmetries in  $\mathbf{C}$ : the tensor product of  $f: X \rightarrow Y$  and  $g: V \rightarrow W$  is given by

$$\left[ \begin{array}{c} x_h \\ \text{---} \end{array} \begin{array}{|c|} \hline f_h \\ \hline \end{array} \begin{array}{c} y_h \\ \text{---} \\ w_h \\ \text{---} \end{array}, \begin{array}{c} v_h \\ \text{---} \end{array} \begin{array}{|c|} \hline g_h \\ \hline \end{array} \begin{array}{c} z_f \\ \text{---} \\ z_g \\ \text{---} \end{array}, \begin{array}{c} z_f \\ \text{---} \\ z_g \\ \text{---} \\ x_a \\ \text{---} \\ v_a \\ \text{---} \end{array} \begin{array}{|c|} \hline f_a \\ \hline \end{array} \begin{array}{|c|} \hline g_a \\ \hline \end{array} \begin{array}{c} y_a \\ \text{---} \\ w_a \\ \text{---} \end{array} \right]$$

i.e., by composing in parallel the honest and adversarial parts and using the swaps to collect the backdoors together. The equivalence relation above is needed so that the interchange law of monoidal categories holds. This concludes our description of the SMC  $\mathbf{C}_{\text{adv}}$ .

There is a functor  $G: \mathbf{C}_{\text{adv}} \rightarrow \mathbf{C}$  sending an object  $(X_h, X_a)$  to  $X_h \otimes X_a$  and a morphism  $[Z, h, a]: (X_h, X_a) \rightarrow (Y_h, Y_a)$  to the composite

$$\begin{array}{c} x_h \\ \text{---} \end{array} \begin{array}{|c|} \hline h \\ \hline \end{array} \begin{array}{c} y_h \\ \text{---} \end{array} \otimes \begin{array}{c} x_a \\ \text{---} \end{array} \begin{array}{|c|} \hline a \\ \hline \end{array} \begin{array}{c} y_a \\ \text{---} \end{array}$$

The functor  $G$  can be seen as giving the single network of ITMs that results from connecting the honest and the adversarial part along the given backdoor.

An interesting sub-SMC of  $\mathbf{C}_{\text{adv}}$  is given by  $\mathbf{C}_{\text{hybrid}}$ , which has the same objects but whose morphisms are constrained by requiring the adversarial part  $a$  to be permutation of wires. This has the effect of trivializing the adversary, letting us talk about networks with an explicit designated backdoor for the adversary without the adversary being present. Finally, one might want to restrict further to some subcategory  $\mathbf{C}_{\text{prot}} \subset \mathbf{C}_{\text{hybrid}}$ , intuitively consisting of machines that have particular vulnerabilities. For example, an incorruptible network could be modeled by a morphism in  $\mathbf{C}_{\text{hybrid}}$  where the adversary’s interface is trivial ( $= I$ ), but one might want one to restrict to a sub SMC  $\mathbf{C}_{\text{prot}}$  which not only lacks such incorruptible behaviors, but all machines involved have some fixed vulnerability. For instance, in UC it is common to use machines which can be fully corrupted by the adversary, relinquishing total control to the adversary once corrupted. Alternatively, one might have your basic building blocks be machines which only leak all information they have to the adversary, modeling honest-but-curious behaviour.

Going forward, our theory is almost built from the SMCs  $\mathbf{C}_{\text{prot}} \subset \mathbf{C}_{\text{hybrid}} \subset \mathbf{C}_{\text{adv}}$ . However, we need additional flexibility to model some idiosyncrasies of UC. These are captured by allowing one to choose a subcategory  $\mathbf{D}_{\text{adv}} \subset \mathbf{C}_{\text{adv}}$  capturing any further requirements one may need. In the case of UC, the main added requirement here is that every machine has exactly one backdoor channel for the adversary. We then set  $\mathbf{D}_{\text{hybrid}} = \mathbf{C}_{\text{hybrid}} \cap \mathbf{D}_{\text{adv}}$  and insist that  $\mathbf{C}_{\text{prot}} =: \mathbf{D}_{\text{prot}}$  was chosen so that  $\mathbf{D}_{\text{prot}} \subset \mathbf{D}_{\text{hybrid}}$ . The theory is then developed relative to  $\mathbf{D}_{\text{prot}} \subset \mathbf{D}_{\text{hybrid}} \subset \mathbf{D}_{\text{adv}}$ .

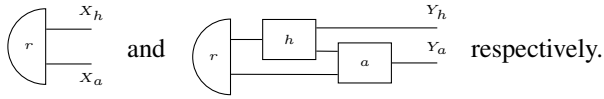
### C. Functionalities and security

We now assume that our SMCs  $\mathbf{D}_{\text{prot}} \subset \mathbf{D}_{\text{hybrid}} \subset \mathbf{D}_{\text{adv}}$  have been fixed. We now define what we mean by resources to

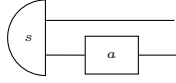
these categories. Intuitively, these correspond to *functionalities* in cryptography. Formally, these will be the objects of the category we will build in Theorem III.8.

**Definition III.2.** A *resource* is a state  $I \rightarrow X$  of  $\mathbf{D}_{\text{hybrid}}$ . An *attacked resource* is a state  $I \rightarrow X$  of  $\mathbf{D}_{\text{adv}}$ .

Formally, both of these are equivalence classes of the form  $[Z, r: I \rightarrow X_h \otimes Z, a: Z \otimes I \rightarrow X_a]$ . However, for  $\mathbf{D}_{\text{hybrid}}$  the morphism  $a$  is permutation of wires and can be ignored for simplicity. This is because one can show that a given kind of resource is always equivalent to one in which the permutation is simply the identity, in the sense that one can always obtain one resource from the other securely. As a result, we will think of these concretely as networks of ITMs, with the remaining tapes split into those belonging to the honest party and the adversary. A resource and the action of a morphism  $[Z, h, a]$  of  $\mathbf{D}_{\text{adv}}$  on such a resource is depicted by



In our reconstruction of UC4SS, the goal in composable cryptography is to use some resources to build other resources, by means of faulty machines i.e., morphisms in  $\mathbf{D}_{\text{prot}}$ . However, such constructions need to be secure (defined formally below) in the sense that they need to withstand attacks by the adversary, requiring the notion of an attacked resource, where an adversary is present. In general, an attacked resource decomposes as



where  $s$  is some network of the honest parties and  $a$  is the action of the adversary. Attacked resources like the above appear when one considers the result of an attack by the adversary (or the result of attaching a simulator).

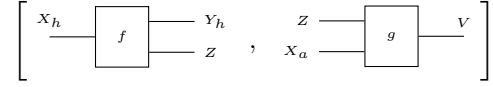
Before giving our security definition, we need one further ingredient, namely, an abstract notion of indistinguishability. In the case of UC, this will coincide with the usual notion of computational indistinguishability (defined formally in section IV-C). For now, all we need to know about computational indistinguishability is that it gives, for each object  $X$  of  $\mathbf{D}_{\text{adv}}$  an equivalence relation  $\approx$  on states  $I \rightarrow X$  in  $\mathbf{D}_{\text{adv}}$ , and that these equivalence relations behave well with respect composition in the sense defined below.

**Definition III.3.** Assume we have an equivalence relation  $\approx$  on states  $I \rightarrow X$  in  $\mathbf{D}_{\text{adv}}$  for every object  $X$ . We say that  $\approx$  *respects composition* if

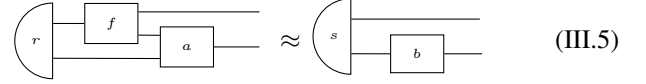
- 1) If  $r \approx s$  for  $r, s: I \rightarrow X$  in  $\mathbf{D}_{\text{adv}}$  and  $f: X \rightarrow Y$  is a morphism in  $\mathbf{D}_{\text{adv}}$ , then  $f \circ r \approx f \circ s$ .
- 2) If  $r_i, s_i: I \rightarrow X_i$  are states in  $\mathbf{D}_{\text{adv}}$  for  $i = 1, 2$  such that  $r_i \approx s_i$ , then  $r_1 \otimes r_2 \approx s_1 \otimes s_2$ .

We now give our security definition which follows the simulation paradigm by quantifying over all possible actions of the adversary.

**Definition III.4.** Let  $r: I \rightarrow X$  and  $s: I \rightarrow Y$  be resources. A morphism  $[Z, f, g]: (X_h, X_a) \rightarrow (Y_h, V)$  of  $\mathbf{D}_{\text{prot}}$



is a *secure emulation*  $r \rightarrow s$  if for every  $a: Z \otimes X_a \rightarrow W$  in  $\mathbf{C}$  (corresponding to an adversary) such that  $[Z, f, a]$  is in  $\mathbf{D}_{\text{adv}}$ , there exists a  $b: V \rightarrow W$  in  $\mathbf{C}$  (corresponding to a simulator) such that  $[Z, f, b]$  is in  $\mathbf{D}_{\text{adv}}$  and

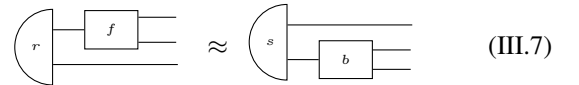


Note that this is more or less the picture people would usually draw when defining simulation-based security, except that (i) it now is a rigorous string diagram and (ii) the  $\approx$  implicitly quantifies over all environments, and as a result the environment does not get drawn.

When  $[Z, f, a]$  lives in the subcategory  $\mathbf{D}_{\text{prot}}$  we can without loss of generality,<sup>2</sup> consider morphisms where the adversary's permutation  $a$  is trivial. Thus, given resources  $r: I \rightarrow X$  and  $s: I \rightarrow Y$  and a morphism  $f: X_h \rightarrow Y_h \otimes Z$  in  $\mathbf{C}$  giving a morphism  $[Z, f, \text{id}]$  in  $\mathbf{D}_{\text{prot}}$ , we will abuse notation and speak of  $f$  being a secure emulation  $r \rightarrow s$  to mean that  $[Z, f, \text{id}]$  in  $\mathbf{D}_{\text{prot}}$  is a secure emulation  $r \rightarrow s$ .

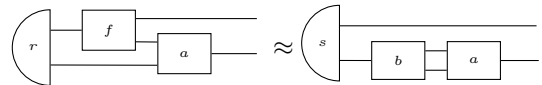
There is a convenient special case which is sufficient.

**Theorem III.6.** [Completeness of the dummy adversary] Assume  $f$  satisfies



for some  $b$ . Then  $f$  is a secure emulation  $r \rightarrow s$ .

*Proof.* Let us assume (III.7). Then



as  $\approx$  respects composition, giving us (III.5).  $\square$

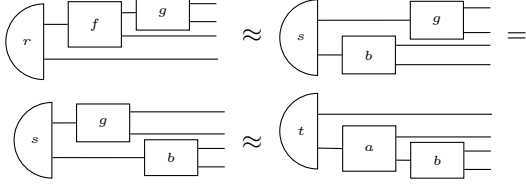
#### D. Categorical composition theorems

For us, the core of the composition theorems is that secure emulation maps form an SMC: this captures both that secure maps are closed under sequential and parallel composition, but also that these compositions are suitably well-behaved. In our opinion, this is what Abstract Cryptography gestures at when talking about “composition-order invariance.” [14, Definition 13]. We now give our composition theorem:

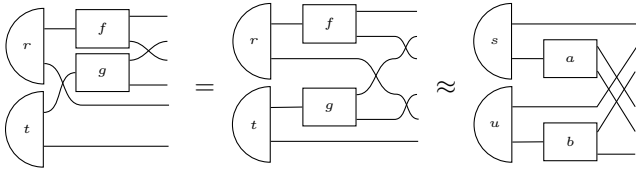
**Theorem III.8.** Resources and secure emulations between them form a symmetric monoidal category.

<sup>2</sup>It is apparent from Definition III.4 that  $[Z, f, a]$  is secure iff  $[Z, f, \text{id}]$  is, so we may restrict our attention to the latter.

*Proof.* We first observe that secure emulations are closed under sequential composition. Assuming  $f: r \rightarrow s$  and  $g: s \rightarrow t$  are secure, so is  $g \circ f: r \rightarrow t$ :



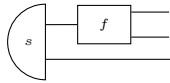
where the first (third) step follows from security of  $f$  (of  $g$ ) and  $\approx$  respecting composition, and the second equation follows from the interchange law. To show that secure maps are closed under parallel composition, we let  $f: r \rightarrow s$  and  $g: t \rightarrow u$  be secure and compute as follows:



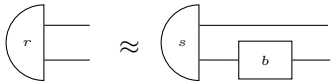
The first equation is true in any monoidal category, and the second equation follows from security of  $f$  and  $g$  and from  $\approx$  respecting composition. It is easy to show that swaps, and in fact all isomorphisms, are secure. Moreover, as our SMC structure is inherited from the SMC  $\mathbf{C}_{\text{adv}}$ , the further required properties of an SMC automatically hold.  $\square$

We now move on to a categorical counterpart of the composition theorem of UC (for static systems) [2, Theorem 3].

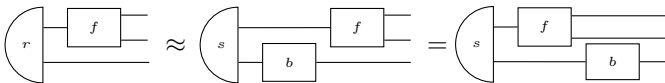
**Theorem III.9.** *Let  $r: I \rightarrow X$  and  $s: I \rightarrow Y$  be resources. Assume that the identity on  $X$  gives a secure emulation  $r \rightarrow s$ . Then for any  $f: X_1 \rightarrow W_1 \otimes Z$  in  $\mathbf{C}$  inducing a map  $[Z, f, \text{id}]$ ,  $f$  gives a secure emulation from  $r$  to*



*Proof.* By assumption there exists a simulator  $b$  such that



It then follows that



which is sufficient by Theorem III.6.  $\square$

#### IV. UC CONCRETELY

We now instantiate our theory in order to capture UC4SS, following the template from the previous section.

##### A. The starting category for UC

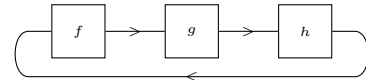
We will now describe how to build by generators and relations an SMC whose morphisms are networks of interactive Turing machines (ITMs). We will fix further details of our machines as needed: for now, it's sufficient to assume that, in addition to some internal tapes, a given ITM has some number  $m$  of tapes for receiving inputs from other ITMs (from now on, input tapes), and some number  $n$  of tapes for receiving subroutine-outputs from other ITMs (from now on, subroutine-output tapes). Additionally, machines will have backdoor tapes, but they do not concern us in this section. We assume that these tapes are enumerated, so that one can always speak without ambiguity of the  $i$ th input tape (or  $j$ th subroutine-output tape). Connections between ITMs are then obtained by pairing the  $i$ th input tape of one machine to the  $j$ th subroutine-output tape together, letting each machine send messages that are received by the other party at the corresponding tape. Thus, as in UC, the tapes are only used for receiving messages. In a formal sense, this pairing of tapes takes place in the SMC we will build, and its operational interpretation is given later in Section IV-C. However, the picture to keep in mind is that when such a pairing is done, one machine can receive inputs from the other machine on its  $i$ th input tape, and can send subroutine outputs to the other machines  $j$ th subroutine-output tape.

As a first approximation, UC has only one type of wire, representing an untyped communication channel. However, the default kind of communication between ITMs in UC is never between equals; given two communicating machines one is always designated to be the “caller” and the other the “subroutine”. We will thus set  $\Sigma := \{A_{\rightarrow}, A_{\leftarrow}\}$ , with  $A_{\rightarrow}$  representing a connection with caller on the right and  $A_{\leftarrow}$  representing a connection with the caller on the left. Instead of labeling the pictures with  $A_{\rightarrow}$  and  $A_{\leftarrow}$  for these two objects, we will simply draw corresponding identity morphisms by an arrow pointing towards the caller, as follows.

$$\longrightarrow \text{ for } \text{id}_{A_{\rightarrow}} \quad \text{ and } \quad \longleftarrow \text{ for } \text{id}_{A_{\leftarrow}}$$

We define  $\Gamma_{A_{\rightarrow}, A_{\leftarrow}}^m, n$  to consist of the set of ITMs with  $m$  subroutine-output tapes and  $n$  input tapes.

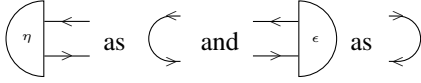
If these were all the generators we had, we could only build DAGs of ITMs whereas UC imposes no restrictions on the shape of connectivity. For example, one could have a cycle of calls, consisting of three machines  $f, g, h$ , with  $h$  calling  $g$ ,  $g$  calling  $f$  and  $f$  calling  $h$ , resulting in a picture like



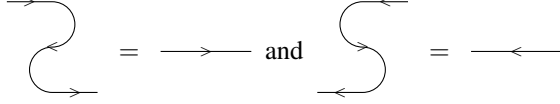
To make sense of this, we need the ability to achieve such connectivity. Formally, we add two additional generating morphisms by setting  $\Gamma_{I, A_{\leftarrow} \otimes A_{\rightarrow}} = \{\eta\}$  and  $\Gamma_{A_{\leftarrow} \otimes A_{\rightarrow}, I} = \{\epsilon\}$  and the defining equations

$$\begin{aligned} (\epsilon \otimes \text{id}_{A_{\rightarrow}}) \circ (\text{id}_{A_{\rightarrow}} \otimes \eta) &= \text{id}_{A_{\rightarrow}} \\ (\text{id}_{A_{\leftarrow}} \otimes \epsilon) \circ (\eta \otimes \text{id}_{A_{\leftarrow}}) &= \text{id}_{A_{\leftarrow}}. \end{aligned}$$

This is demystified by drawing



The defining equations then become the following *snake equations*:



Setting all other generating sets  $\Gamma_{V,W}$  to be empty then results in an SMC which we'll denote by  $\mathbf{B}$ .

### B. Protocols, hybrids, and the adversary for in UC

Our starting category  $\mathbf{B}$  yields categories  $\mathbf{B}_{\text{adv}}$  and  $\mathbf{B}_{\text{hybrid}}$  as in Section III-B. However, we now need to cut down from  $\mathbf{B}_{\text{adv}}$  to certain subcategories in order to model UC faithfully. We will introduce nested sub-SMCs  $\mathbf{UC}_{\text{adv}} \supset \mathbf{UC}_{\text{hybrid}} \supset \mathbf{UC}_{\text{prot}}$  of  $\mathbf{B}_{\text{adv}}$ .

In each of these the objects are restricted to pairs  $(X_h, X_a)$  of objects of  $\mathbf{B}$  where both  $X_h$  and  $X_a$  are words in the generating object  $A_{\rightarrow}$  of  $\mathbf{B}$  (instead of  $\{A_{\rightarrow}, A_{\leftarrow}\}$ ): this is to ensure that states in the resulting category are given by networks of ITMs that are subroutines of the environment, ruling out states that also have machines calling the environment. In other words, every object is of the form  $(A_{\rightarrow}^m, A_{\rightarrow}^n)$  for some  $m, n \in \mathbb{N}$ . The interpretation is that  $m$  is the number of main machines and  $n$  is the number of adversarial backdoor tapes. As the other object is not referenced from now on, we will simply write  $A$  instead of  $A_{\rightarrow}$ . Moreover, we will impose the following additional restrictions on all morphisms  $[Z, f, g]$ .

- 1) For  $\mathbf{UC}_{\text{adv}}$ , we want to ensure that every machine that is part of the network of honest machines  $f$  has exactly one tape for the adversary. This is achieved by setting  $\mathbf{UC}_{\text{adv}}$  to be the sub-SMC of  $\mathbf{B}_{\text{adv}}$  generated by (i.e., the smallest sub-SMC containing) all morphisms of the form
  - $[A, f: A^m \rightarrow A^n \otimes A, \text{id}]$ , where  $f$  consists of a single ITM with  $m$  subroutine-output tapes and  $n + 1$  input tapes, and
  - $[Z, \text{id}, g]$  (some honest wires, arbitrary adversaries)
- 2) We set  $\mathbf{UC}_{\text{hybrid}} := \mathbf{B}_{\text{hybrid}} \cap \mathbf{UC}_{\text{adv}}$ .
- 3) For  $\mathbf{UC}_{\text{prot}}$  we additionally require that all of the allowed ITMs behave in some fixed prescribed way (e.g., giving full control and all of its history to the adversary) when instructed by the adversary. This is achieved by setting  $\mathbf{UC}_{\text{prot}}$  to be the sub-SMC of  $\mathbf{UC}_{\text{hybrid}}$  generated by all morphisms of the form
  - $[A, f: A^m \rightarrow A^n \otimes A, \text{id}]$ , where  $f$  consists of a single ITM with  $m$  subroutine-output tapes and  $n+1$  input tapes and *moreover*, when instructed to do so at the  $n + 1$ st tape,  $f$  gives full control to the machine it is connected to, and
  - $[Z, \text{id}, g]$  where  $g$  is a permutation of wires

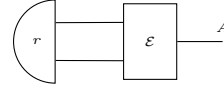
### C. Execution model and computational indistinguishability

We now fill in the details of computational indistinguishability. First, we spell out some further details of ITMs.

- First of all, we allow our ITMs to be probabilistic, which can be modeled by each of them having access to an internal randomness tape, containing a uniformly random infinite sequence of bits (of which any machine will consume only a finite amount).
- Machines are poly-time in the security parameter. Generally speaking, choosing a good notion of efficiency is surprisingly subtle [44]. For us, what matters is there is a notion of efficiency for machines and networks such that
  - Any network of efficient machines is an efficient network.
  - Any efficient network can be replaced by an efficient machine simulating the network that is indistinguishable from it. (This justifies allowing the adversary to consist of multiple machines)

For Theorem IV.3 to hold, our notion of efficiency needs to agree with that of UC4SS, which in turn also relies on the above properties.

We now define, for each object  $X = (X_h, X_a)$  of  $\mathbf{UC}_{\text{adv}}$ , the equivalence relation of computational indistinguishability on states  $I \rightarrow X$  of  $\mathbf{UC}_{\text{adv}}$ . First, we define an *environment* of type  $X$  to be a morphism  $\mathcal{E}: X_h \otimes X_a \rightarrow A$  in  $\mathbf{B}$ , i.e., a network of ITMs with one input tape and subroutine-output tapes given by  $X_h \otimes X_a$ , with some further constraints explained below. Now, given a state  $r: I \rightarrow X$  in  $\mathbf{UC}_{\text{adv}}$ , we can first apply the functor  $G: \mathbf{UC}_{\text{adv}} \rightarrow \mathbf{B}$  to it, obtaining a state  $G(r): I \rightarrow X_a \otimes X_h$  in  $\mathbf{B}$ , and then feed this state into the environment, resulting in  $\mathcal{E} \circ G(r)$ :

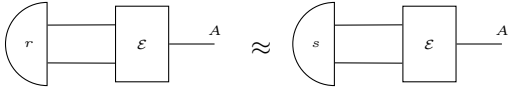


We'll now explain how this is to be executed when  $\mathcal{E}$  is given an initial input  $z$ , following the single-threaded execution model of UC4SS. Initially, the environment  $\mathcal{E}$  receives the input  $z$  (whose length gives the security parameter) on the machine for which the only input tape of  $\mathcal{E}$  belongs, and this *initiating machine* of  $\mathcal{E}$  is then activated. After that, the machines take turns being executed as follows: the currently active machine is executed until it either sends a message to another machine, after which the message is written on the corresponding tape of the recipient machine which now becomes active, or the machine halts without sending a message. If the machine that halted was not the initiating machine of  $\mathcal{E}$ , we activate the initiating machine of  $\mathcal{E}$  again. If it was the initiating machine of  $\mathcal{E}$ , the whole execution is finished. We also assume that  $\mathcal{E}$  never halts without sending a message outside of it (whether as input on its left or as subroutine-output on its right), and that the subroutine-output message sent by  $\mathcal{E}$  is always just a binary value  $\{0, 1\}$ . Thus the computation goes back and forth between  $\mathcal{E}$  and  $r$  until  $\mathcal{E}$  concludes by returning a bit  $b \in \{0, 1\}$ .

For any state  $r$ , input  $z$  and  $\mathcal{E}$ , we thus obtain a probability distribution  $\text{EXEC}_{\mathcal{E},r}(z)$  on  $\{0,1\}^*$ , where the randomness comes from internal randomness used by  $\mathcal{E}$  and  $r$ . We thus obtain a probability ensemble  $\{\text{EXEC}_{\mathcal{E},r}(z)\}_{z \in \{0,1\}^*}$ , which we denote by  $\text{EXEC}_{\mathcal{E},r}$ . We say that  $r, s: I \rightarrow X$  in  $\text{UC}_{\text{adv}}$  are *computationally indistinguishable*, denoted  $r \approx s$ , if for any environment  $\mathcal{E}$  of type  $X$ , the (statistical) difference between the ensembles  $\text{EXEC}_{\mathcal{E},r}$  and  $\text{EXEC}_{\mathcal{E},s}$  (i.e., its distinguishing advantage) is negligible in  $|z|$ . Note that in our notation the adversary (resp. simulator) is subsumed into  $r$  (resp.  $s$ ). We now observe that computational indistinguishability behaves well with composition.

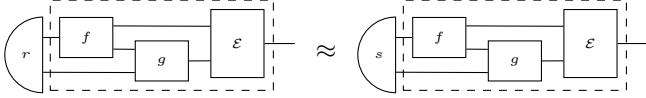
**Lemma IV.1.** *Computational indistinguishability respects composition*

*Proof.* Let us overload notation as follows: given  $r, s: I \rightarrow X$  and an environment  $\mathcal{E}$  of the correct type, we will write



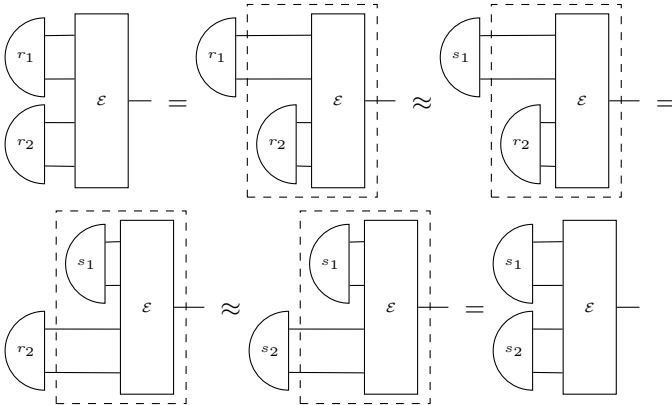
to mean that the (statistical) difference between the ensembles  $\text{EXEC}_{\mathcal{E},r}$  and  $\text{EXEC}_{\mathcal{E},s}$  is negligible.

With this notation, the proof is just a matter of absorbing parts to the environment, denoted by dashed lines. Let  $r, s: I \rightarrow X$  be states in  $\text{UC}_{\text{adv}}$ , and let  $[Z, f, g]: X \rightarrow Y$  be a morphism in  $\text{UC}_{\text{adv}}$ . Let us assume  $r \approx s$ . In order to show that  $[Z, f, g] \circ r \approx [Z, f, g] \circ s$ , we let  $\mathcal{E}$  be an arbitrary environment and compute as follows:



where the (implicit) steps of adding and removing dashed areas follow from associativity of composition, and the displayed step follows from the assumption that  $r \approx s$ .

For parallel composition, let  $r_i, s_i: I \rightarrow X_i$  be states in  $\text{UC}_{\text{adv}}$  for  $i = 1, 2$  such that  $r_i \approx s_i$ . Then  $r_1 \otimes r_2 \approx s_1 \otimes s_2$ . Let  $\mathcal{E}$  be arbitrary. We then reason as follows:



□

Observe that this Lemma does once and for all the steps of absorbing parts of the network into the environment. Given the

diagrammatic proof, the Lemma holds in fact more generally, but we omit the precise general statement.

Now that we have the subcategories  $\text{UC}_{\text{prot}} \subset \text{UC}_{\text{hybrid}} \subset \text{UC}_{\text{adv}}$  and a composition-respecting equivalence relation, the theory in Section III readily applies. In particular, Theorems III.8 and III.9 give rise to composition theorems for this specific model. We will now discuss how this model relates to UC4SS.

#### D. Relationship to and differences with UC

We now discuss how our setting differs in some technical details from that of UC, despite being in essence the same theory. First of all, for us, an ITM can have multiple communication tapes. In contrast, in UC every machine has only one input tape and one subroutine-output tape (and one backdoor tape).

These differences affect how one captures the connectivity of a network of Turing machines. In our setting, we explicitly add data keeping track of which tapes are paired together. In contrast, in UC each machine is modeled as a triple  $\mu = (\text{ID}_\mu, C_\mu, \tilde{\mu})$ , where  $\text{ID}_\mu$  is the *identity of a machine*, i.e., a string giving the machine its name,  $C_\mu$  is its *communication set*, i.e., a set of pairs of the form  $(b, \text{ID})$ , where  $\text{ID}$  is the name of a machine and  $b \in \{\text{input, subroutine-output, backdoor}\}$ , and finally  $\tilde{\mu}$  is the actual program of the machine.

For a set  $\pi$  of such machines to form a *protocol*, the following conditions must be satisfied.

- The identities of the machines in  $\pi$  are distinct from each other and from  $\{0,1\}$  (as these are restricted for the environment and the adversary).
- Backdoor communication is not used,<sup>3</sup> i.e., every entry  $(b, \text{ID})$  appearing in the communication set of some machine in  $\pi$  has  $b \in \{\text{input, subroutine-output}\}$ .
- If  $(\text{input}, \text{ID})$  is in the communication set of some machine  $\mu \in \pi$  with identity  $\text{ID}_\mu$ , then there is a machine in  $\pi$  whose identity is  $\text{ID}$  and whose communication set contains  $(\text{subroutine-output}, \text{ID}_\mu)$ .
- If  $(\text{subroutine-output}, \text{ID})$  is in the communication set of some machine  $\mu$  in  $\pi$  and  $\text{ID}$  is the identity of some machine in  $\pi$ , then that machine's communication set contains  $(\text{input}, \text{ID}_\mu)$ .

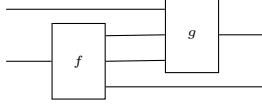
In the last condition above, if  $\text{ID}$  is not the identity of any machine in  $\pi$ , we say that  $\mu$  is a *main machine* of  $\pi$  and  $\text{ID}$  is an *external identity* of  $\pi$ .

The final difference is that in UC, the adversary is always a single machine, whereas for us the adversary is allowed to be a network of machines. This difference is not essential, as we can replace a network of adversaries with a single adversary simulating the network, without affecting efficiency.

We now want to argue that the other differences do not matter either, by giving a formal translation between our setting and UC4SS. We first note that the possible kinds of connectivity do slightly differ between the two approaches.

<sup>3</sup>Formally, there are no such restrictions on protocols in [2, Section 2]. However, we believe this is an oversight.

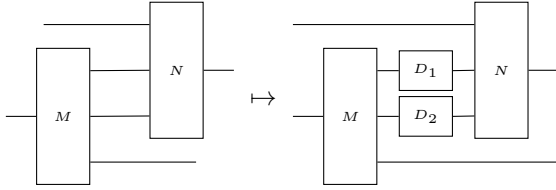
The main difference is that our formalism allows two (or more) tapes of one machine to be connected more than once to itself, another machine in the protocol, or the environment, with an example of two machines  $f$  and  $g$  being connected twice given by



Whenever a state involves machines that are wired together more than once, we say that the state has *multiple parallel connections*. We also say this when there's at least two connections left open from a single machine, as these will result in such connections with the environment. We first observe that we can remove all multiple parallel connections.

**Lemma IV.2.** *Let  $r$  be an (attacked) resource. Then there is a resource  $s$  without multiple parallel connections that is computationally indistinguishable from  $r$ .*

*Proof.* Start from  $r$ , and add “dummy machines” that simply pass all messages through to remove all multiple parallel connections. This is illustrated by



□

We are now in a position to translate from our formalism to UC and back. However, as our machines do not have names but the ones in UC do, such a naming must be provided by the translation. While any naming convention would do, the translation works well when the naming is done similarly for all states. Recall that in UC,  $ID = 0$  is reserved for the environment, and 1 is reserved for the adversary.

**Theorem IV.3.** *Let  $M$  and  $N$  be disjoint sets of  $m$  and  $n$  identities of machines, respectively, with  $M$  not containing 1 and  $N$  not containing 0 or 1. Fix bijections  $h: \{1, \dots, m\} \rightarrow M$  and  $k: \{1, \dots, n\} \rightarrow N$ .*

- 1) *There is a mapping  $F := F_{h,k}$  sending any resource  $(I, I) \rightarrow (A^m, A^n)$  in  $\mathbf{UC}_{\text{hybrid}}$  without multiple parallel connections to a protocol in UC whose set of machine identities is  $N$  and whose set of external identities is  $M$ . There is also a mapping  $G := G_{h,k}$  in the opposite direction.*
- 2) *The mapping  $F$  extends to a mapping sending any (attacked) resource  $(I, I) \rightarrow (A^m, A^n)$  in  $\mathbf{UC}_{\text{adv}}$  without multiple parallel connections into a hybrid protocol (with adversary), where  $N$  gives the names of the honest machines and ideal functionalities, and  $M$  is the set of external identities. Similarly,  $G$  extends in the opposite direction.*

- 3) *For any UC-protocol  $\pi$  (potentially with an adversary),  $FG(\pi) \approx \pi$ , and for any (attacked) resource  $r$  in  $\mathbf{UC}_{\text{adv}}$ ,  $GF(r) \approx r$ .<sup>4</sup>*
- 4) *These translations extend to the environment, and the resulting probability ensembles  $\text{EXEC}_{\mathcal{E},r}$  and  $\text{EXEC}_{F(\mathcal{E}),F(r)}$  are equal, and similarly for the translation  $G$ . As a consequence,  $F$  and  $G$  preserve computational indistinguishability, and  $g: r \rightarrow s$  is secure iff  $F(g \circ r)$  UC-emulates  $s$ .*

*Proof.* 1) We begin by defining  $F$ . Let  $f_i$  be the machine that the  $i$ th adversarial port connects to. We will equip  $f_i$  with the name  $k(i)$ , and read off the communication sets from the connectivity of the state. As we have no multiple parallel connections, this is straightforward: we use the names provided above and the type of connection in  $\{\text{input}, \text{subroutine-output}\}$  is read from the object  $\in \{A, B\}$  corresponding to the wire. We now deal with wires that are not connected to anything: for the  $j$ th such wire, we add  $(h(j), \text{subroutine-output})$  to the communication set. We also adjust the code of each machine slightly: when a machine gets a message  $(ID \in M \cup N, \text{msg})$ , it behaves as the original machine did when getting the message  $\text{msg}$  on the wire connected to  $ID$ .

For  $G$ , we can read off the connections from the communication sets, letting  $\mu_i$  be the machine named  $k(i)$ . We adjust the behavior of the machines as follows: when a machine  $\mu_i$  gets a message  $\text{msg}$  on a wire connected to a machine with identity  $ID \in M \cup N$ , it behaves as the original machine did when receiving message  $(ID, \text{msg})$ . Moreover, for each machine, we add one more wire connected to the adversarial part: when receiving a message there, the machine will behave as it did when receiving messages via its backdoor connection.

What's left to specify is ordering of the outgoing ports: this is done using the order induced by the bijection  $h$ , so that the first wire in  $A^m$  connects to the first machine that is a main identity and so on.

- 2) For  $F$ , we proceed as above, except we first replace the network of adversarial machines by a single (equivalent) machine which we name 1, and add  $(1, \text{backdoor})$  to the communication set of every honest machine. For  $G$ , we replace all backdoor connections with a wire of type  $A$ , and otherwise proceed as above.
- 3) It is clear from the construction.

4) Any environment can be translated along  $F$  and  $G$  similarly, and by construction, the resulting probability ensembles are equal. It then follows that if  $r$  and  $s$  are computationally distinguishable, so are  $F(r)$  and  $F(s)$ , and similarly for  $G$ . Hence  $F$  and  $G$  are injective on indistinguishability classes. That  $F$  and  $G$  also preserve indistinguishability follows from the following general fact: If  $(X, \approx_X)$  and  $(Y, \approx_Y)$  are two sets equipped with an equivalence relations, and  $F: X \rightarrow Y$  and  $G: Y \rightarrow X$  are functions that are injective on equivalence classes satisfying  $GF(x) \approx_X x$  and  $FG(y) \approx_Y y$  for all  $x \in X, y \in Y$ , then  $F$  and  $G$  also preserve the equivalence

<sup>4</sup>In fact, these are perfectly indistinguishable, but we have not defined that notion here.

relations. As both our notion of security and UC-emulation are defined in terms of the existence of a suitable simulator (adversary) satisfying a property in terms of computational indistinguishability, the final claim follows.  $\square$

Using Theorem IV.3 we see that Theorem III.6 translates to “completeness of the dummy adversary” in UC. One can also use Theorem IV.3 to deduce that any UC-network is indistinguishable from a naming-invariant one — going back and forth via the translation will hard-code the names, so that renaming machines no longer affects the behavior.

We now discuss the universal composition theorem of UC4SS [2, Theorem 3]. Let  $\pi, \phi, \rho$  be protocols in UC. Recall:

- $\phi$  is a *subroutine protocol* of  $\rho$  if  $\phi \subset \rho$  as a set of ITMs.
- $\pi$  is *compatible* with  $\phi$  if there is an identity-preserving bijective correspondence between main machines of  $\pi$  and  $\phi$ , and the entries corresponding to external identities in the communication sets of the main machines are equal.
- If  $\phi$  is a subroutine protocol of  $\rho$ , then  $\pi$  is *identity-compatible* with  $\rho$  and  $\phi$  if no machine in  $\pi$  has the same identity as a machine in  $\rho$ .

**Claim IV.4.** Assume now that  $\pi, \phi, \rho$  are protocols,  $\phi$  is a subroutine protocol of  $\rho$ ,  $\pi$  is compatible with  $\phi$  and  $\pi$  is identity-compatible with  $\rho$  and  $\phi$ . Then  $\rho^{\phi \rightarrow \pi} := (\rho \setminus \phi) \cup \pi$  is a protocol.

A similar claim is made in [2, Section 2] but we do not believe it is correct as stated there for the following reasons: When defining compatibility, [2, Section 2] only requires an injective correspondence. However, this is not enough for the claim, since  $\rho \setminus \phi$  might want to talk to a main machine present in  $\phi$  but missing from  $\pi$ . Similarly, [2, Section 2] only requires an inclusion of external identities. However, this is not enough for the claim, since a machine in  $\rho \setminus \phi$  might want to talk to a main machine in  $\phi$  that is missing from the communication set of the corresponding machine in  $\pi$ . Finally, [2, Section 2] only requires identity-compatibility and not compatibility + identity-compatibility, but again this jeopardizes the claim.

**Corollary IV.5** (Essentially [2, Theorem 3]). Assume that  $\pi, \phi, \rho$  are protocols,  $\phi$  is a subroutine protocol of  $\rho$ ,  $\pi$  is compatible with  $\phi$  and  $\pi$  is identity-compatible with  $\rho$  and  $\phi$ . If  $\pi$  UC-emulates  $\phi$ , then  $\rho^{\phi \rightarrow \pi}$  UC-emulates  $\rho$ .

*Proof.* Let us translate  $\pi, \phi, \rho$  along Theorem IV.3. Then compatibility of  $\phi$  and  $\pi$  amounts to saying that their translates give rise to states  $r_\pi: (I, I) \rightarrow (X, Y_\pi)$  and  $s_\phi: (I, I) \rightarrow (X, Y_\phi)$ , so that their honest interfaces agree, and thus  $\rho \setminus \phi$  gets translated to a morphism  $f$  that can be composed with (the honest ports) of either. Therefore Theorem III.9 applies, so that (abusing notation)  $f r_\pi$  emulates  $f s_\phi$ , which translates back to the fact that  $\rho^{\phi \rightarrow \pi}$  UC-emulates  $\rho$ .  $\square$

## V. CONCLUSION AND OUTLOOK

In this work we have initiated a categorical study of UC4SS, giving rigorous diagrammatic proofs of basic results such as the composition theorems. As a result, our theory holds more

generally and gives a template for producing similar theories where ITMs are replaced by other computational systems. Moreover, categorical thinking fixed some prior oversights and resulted in several beneficial changes to UC4SS (doing away with identities of machines altogether, allowing both the environment and the adversary to consist of multiple machines) while remaining inter-translatable with UC4SS.

Our work also opens up fertile grounds for future:

- Techniques used in [26] can readily be used to develop other variants of the theory: for example, we can develop a version where the honest machines are partitioned into  $n$  sets ( $n$  parties), with all communication across the partitions mediated by a shared functionality. We expect that the resulting categorical theory stands in a similar relationship to [6] as our theory here relates to UC4SS. Similarly, we could readily reason about (abstract) distances up to  $\varepsilon > 0$ , rather than reasoning up to indistinguishability.
- While our model allows the adversary to adaptively decide which machines to corrupt during execution, in some sense we do not really give a fully categorical analysis of corruption: one could imagine having the adversary absorb any machine it corrupts and denote this visually in string diagrams. Can one make this precise? As far as we are aware, such dynamically changing string diagrams have not been used in the literature.
- Can we model full UC similarly? Existing categorical tools are not well-suited for studying a dynamically changing network, making this a difficult question.
- We intend to provide a categorical analysis of other frameworks for composability, such as abstract/constructive cryptography [3], [14] and specifically random systems [15], the IITM model [16], and state-separating proofs [17]. This would show that the composition theorem of [26] can serve as “the one composition theorem to rule them all.”
- More importantly, recasting different frameworks under the categorical umbrella would help with comparing different frameworks and pave the way for translating results across frameworks. Indeed, [26, Theorem 4.10] records a situation where symmetric monoidal functors preserve security. In the long term, the goal would be to connect different composability frameworks via a web of functors, thus giving a full answer to when a protocol secure in one framework is secure in others.

## REFERENCES

- [1] R. Canetti, “Universally composable security: A new paradigm for cryptographic protocols,” in *42nd Annual Symposium on Foundations of Computer Science—FOCS 2001*, 2001, pp. 136–145.
- [2] —, “Universally composable security,” *J. ACM*, vol. 67, no. 5, pp. 28:1–28:94, 2020. [Online]. Available: <https://doi.org/10.1145/3402457>
- [3] U. Maurer, “Constructive cryptography—a new paradigm for security definitions and proofs,” in *Joint Workshop on Theory of Security and Applications—TOSCA 2011*, 2011, pp. 33–56.
- [4] R. Küsters, “Simulation-based security with inexhaustible interactive turing machines,” in *19th IEEE Computer Security Foundations Workshop, (CSFW-19 2006)*, 5–7 July 2006, Venice, Italy. IEEE Computer Society, 2006, pp. 309–320. [Online]. Available: <https://doi.org/10.1109/CSFW.2006.30>

- [5] J. Baez and M. Stay, “Physics, Topology, Logic and Computation: A Rosetta Stone,” in *New Structures for Physics*, B. Coecke, Ed. Berlin, Heidelberg: Springer, 2011, pp. 95–172. [Online]. Available: [https://doi.org/10.1007/978-3-642-12821-9\\_2](https://doi.org/10.1007/978-3-642-12821-9_2)
- [6] R. Canetti, A. Cohen, and Y. Lindell, “A simpler variant of universally composable security for standard multiparty computation,” in *Advances in Cryptology - CRYPTO 2015 - 35th Annual Cryptology Conference, Santa Barbara, CA, USA, August 16-20, 2015, Proceedings, Part II*, ser. Lecture Notes in Computer Science, R. Gennaro and M. Robshaw, Eds., vol. 9216. Springer, 2015, pp. 3–22. [Online]. Available: [https://doi.org/10.1007/978-3-662-48000-7\\_1](https://doi.org/10.1007/978-3-662-48000-7_1)
- [7] R. Canetti, A. Stoughton, and M. Varia, “Easyuc: Using easycrypt to mechanize proofs of universally composable security,” in *2019 IEEE 32nd Computer Security Foundations Symposium (CSF)*. IEEE, Jun. 2019, p. 167–16716. [Online]. Available: <http://dx.doi.org/10.1109/CSF.2019.00019>
- [8] C. Badertscher, R. Canetti, J. Hesse, B. Tackmann, and V. Zikas, “Universal composition with global subroutines: Capturing global setup within plain uc,” in *Theory of Cryptography Conference*. Springer, 2020, pp. 1–30.
- [9] S. Mac Lane, *Categories for the Working Mathematician*, 2nd ed. Springer, 1971.
- [10] D. Unruh, “Universally composable quantum multi-party computation,” in *Advances in Cryptology—EUROCRYPT 2010*, 2010, pp. 486–505.
- [11] J. Gancher, K. Sojakova, X. Fan, E. Shi, and G. Morrisett, “A core calculus for equational proofs of cryptographic protocols,” *Proc. ACM Program. Lang.*, vol. 7, no. POPL, pp. 866–892, 2023. [Online]. Available: <https://doi.org/10.1145/3571223>
- [12] X. Fan, J. Gancher, G. Morrisett, E. Shi, and K. Sojakova, “IPDL: A probabilistic dataflow logic for cryptography,” *IACR Cryptol. ePrint Arch.*, p. 1422, 2019. [Online]. Available: <https://eprint.iacr.org/2019/1422>
- [13] K. Liao, M. A. Hammer, and A. Miller, “ILC: a calculus for composable, computational cryptography,” in *Proceedings of the 40th ACM SIGPLAN Conference on Programming Language Design and Implementation, PLDI 2019, Phoenix, AZ, USA, June 22-26, 2019*, K. S. McKinley and K. Fisher, Eds. ACM, 2019, pp. 640–654. [Online]. Available: <https://doi.org/10.1145/3314221.3314607>
- [14] U. Maurer and R. Renner, “Abstract cryptography,” in *Innovations in Computer Science—ICS 2011*, 2011.
- [15] U. Maurer, *Indistinguishability of Random Systems*. Springer Berlin Heidelberg, 2002, p. 110–132. [Online]. Available: [http://dx.doi.org/10.1007/3-540-46035-7\\_8](http://dx.doi.org/10.1007/3-540-46035-7_8)
- [16] R. Küsters, M. Tuengerthal, and D. Rausch, “The IITM model: a simple and expressive model for universal composability,” *Journal of Cryptology*, vol. 33, no. 4, pp. 1461–1584, 2020.
- [17] C. Brzuska, A. Delignat-Lavaud, C. Fournet, K. Kohbrok, and M. Kohlweiss, “State Separation for Code-Based Game-Playing Proofs,” in *Lecture Notes in Computer Science*. Springer International Publishing, 2018, pp. 222–249. [Online]. Available: [https://doi.org/10.1007/978-3-030-03332-3\\_9](https://doi.org/10.1007/978-3-030-03332-3_9)
- [18] B. Pfitzmann and M. Waidner, “A model for asynchronous reactive systems and its application to secure message transmission,” in *2001 IEEE Symposium on Security and Privacy—S&P 2001*, 2000, pp. 184–200.
- [19] M. Backes, B. Pfitzmann, and M. Waidner, “A general composition theorem for secure reactive systems,” in *1st Theory of Cryptography Conference—TCC 2004*, 2004, pp. 336–354.
- [20] —, “The reactive simulatability (RSIM) framework for asynchronous systems,” *Information and Computation*, vol. 205, no. 12, pp. 1685–1720, 2007.
- [21] D. Micciancio and S. Tessaro, “An equational approach to secure multiparty computation,” in *4th Conference on Innovations in Theoretical Computer Science—ITCS 2013*, 2013, pp. 355–372.
- [22] D. Hofheinz and V. Shoup, “GNUC: A new universal composability framework,” *Journal of Cryptology*, vol. 28, no. 3, pp. 423–508, 2015.
- [23] K. Liao, M. A. Hammer, and A. Miller, “ILC: a calculus for composable, computational cryptography,” in *Proceedings of the 40th ACM SIGPLAN Conference on Programming Language Design and Implementation*. ACM, Jun. 2019, pp. 640–654.
- [24] D. Rausch, R. Küsters, and C. Chevalier, *Embedding the UC Model into the IITM Model*. Springer International Publishing, 2022, p. 242–272. [Online]. Available: [http://dx.doi.org/10.1007/978-3-031-07085-3\\_9](http://dx.doi.org/10.1007/978-3-031-07085-3_9)
- [25] A. Broadbent and M. Karvonen, “Categorical composable cryptography,” in *Foundations of Software Science and Computation Structures (FoSSaCS)*, ser. Lecture Notes in Computer Science, vol. 13242. Springer, 2022, pp. 161–183.
- [26] —, “Categorical composable cryptography: extended version,” *Logical Methods in Computer Science*, vol. 19, pp. 30:1–30:46, 2023.
- [27] M. Prabhakaran and M. Rosulek, “Cryptographic complexity of multiparty computation problems: Classifications and separations,” in *Advances in Cryptology—CRYPTO 2008*, 2008, pp. 262–279.
- [28] M. Rosulek, *The Joy of Cryptography: An Undergraduate Course in Provable Security*. MIT Press, 2026.
- [29] C. Brzuska and S. Oechsner, “A State-Separating Proof for Yao’s Garbling Scheme,” in *2023 IEEE 36th Computer Security Foundations Symposium (CSF)*. IEEE, 2023, pp. 137–152.
- [30] S. Breiner, C. A. Miller, and N. J. Ross, “Graphical methods in device-independent quantum cryptography,” *Quantum*, vol. 3, p. 146, 2019.
- [31] B. Coecke, Q. Wang, B. Wang, Y. Wang, and Q. Zhang, “Graphical calculus for quantum key distribution (extended abstract),” *Electronic Notes in Theoretical Computer Science*, vol. 270, no. 2, pp. 231–249, 2011.
- [32] S. Breiner, A. Kalev, and C. A. Miller, “Parallel self-testing of the GHZ state with a proof by diagrams,” in *Proceedings of QPL 2018*, ser. Electronic Proceedings in Theoretical Computer Science, vol. 287, 2018, pp. 43–66.
- [33] A. Hillebrand, “Superdense coding with GHZ and quantum key distribution with W in the ZX-calculus,” in *Proceedings of QPL 2011*, ser. Electronic Proceedings in Theoretical Computer Science, vol. 95, 2011, pp. 103–121.
- [34] M. Stay and J. Vicary, “Bicategorical semantics for nondeterministic computation,” in *Proceedings of the Twenty-ninth Conference on the Mathematical Foundations of Programming Semantics, MFPS XXIX*, ser. Electronic Notes in Theoretical Computer Science, vol. 298, 2013, pp. 367 – 382.
- [35] L. Colisson, D. Markham, and R. Yehia, “All graph state verification protocols are composable secure,” 2024.
- [36] P. Selinger, “A survey of graphical languages for monoidal categories,” in *New structures for physics*. Springer, 2010, pp. 289–355.
- [37] R. Pedeletu and F. Zanasi, *An Introduction to String Diagrams for Computer Scientists*, ser. Elements in Applied Category Theory. Cambridge University Press, 2025.
- [38] B. Coecke and E. O. Paquette, “Categories for the practising physicist,” in *New Structures for Physics*. Springer, 2010, pp. 173–286.
- [39] B. Fong and D. I. Spivak, *An Invitation to Applied Category Theory: Seven Sketches in Compositionality*. Cambridge University Press, 2019.
- [40] C. Heunen and J. Vicary, *Categories for Quantum Theory: an introduction*. Oxford University Press, USA, 2019.
- [41] S. Awodey, *Category theory*. Oxford University Press, 2010.
- [42] T. Leinster, *Basic category theory*. Cambridge University Press, 2014, vol. 143.
- [43] M. Bartoletti, I. Castellani, P.-M. Deniérou, M. Dezani-Ciancaglini, S. Ghilezan, J. Pantovic, J. A. Pérez, P. Thiemann, B. Toninho, and H. T. Vieira, “Combining behavioural types with security analysis,” *Journal of Logical and Algebraic Methods in Programming*, vol. 84, no. 6, pp. 763–780, Nov. 2015. [Online]. Available: <https://www.sciencedirect.com/science/article/pii/S2352220815000851>
- [44] D. Hofheinz, D. Unruh, and J. Müller-Quade, “Polynomial runtime and composability,” *Journal of Cryptology*, vol. 26, no. 3, p. 375–441, Aug. 2012. [Online]. Available: <http://dx.doi.org/10.1007/s00145-012-9127-4>

## APPENDIX A

### COMPOSITION THEOREMS AS AN INSTANCE OF [26]

In this section we sketch briefly how our theory fits (a minor variant of) the general theory put forward in [25], [26]. For that, we need to recall briefly the main theory of [25], [26]. We will do so while assuming the notions of a strong (or lax) symmetric monoidal functor. The general setting is given by specifying

- a strong symmetric monoidal functor  $F: \mathbf{D} \rightarrow \mathbf{C}$ , which intuitively can be seen as the mapping interpreting (honest) protocols into more general behaviors.

- a lax monoidal functor  $R: \mathbf{C} \rightarrow \mathbf{Set}$ , which intuitively gives for any object  $A$  of  $\mathbf{C}$  the set  $R(A)$  of resources of type  $A$ , and maps morphisms of  $\mathbf{C}$  into their actions on resources.
- an attack model  $\mathcal{A}$  on  $\mathbf{C}$ : this specifies how the adversary can attack a protocol, and security relative to  $\mathcal{A}$  is then defined following the simulation paradigm.

The composition theorem given in [26, Theorem 4.6] then ensures that secure morphisms form an SMC, and in particular that security is closed under secure and parallel composition. Theorem III.9 is almost an instance of this: we just need to tweak the framework of [26] slightly. The main change is that we no longer formulate correctness of protocols as an additional correctness equation, and have only the security conditions. Moreover, we need to observe that the attacker in our setting behaves in a particularly convenient way, which is formalized in Lemma A.3. We now formally give the abstract definition of an attack model and security with respect to it.

**Definition A.1.** An *attack model*  $\mathcal{A}$  on an SMC  $\mathbf{C}$  consists of giving for each morphism  $f$  of  $\mathbf{C}$  a class  $\mathcal{A}(f)$  of morphisms of  $\mathbf{C}$  such that

- $f \in \mathcal{A}(f)$  for every  $f$ .
- For any  $f: A \rightarrow B$  and  $g: B \rightarrow C$  and  $g' \in \mathcal{A}(g), f' \in \mathcal{A}(f)$  that are composable, we have  $g' \circ f' \in \mathcal{A}(g \circ f)$ . Moreover, any  $h \in \mathcal{A}(g \circ f)$  factorizes as  $g' \circ f'$  with  $g' \in \mathcal{A}(g)$  and  $f' \in \mathcal{A}(f)$ .
- For any  $f: A \rightarrow B, g: C \rightarrow D$  in  $\mathbf{C}$  and  $f' \in \mathcal{A}(f), g' \in \mathcal{A}(g)$  we have  $f' \otimes g' \in \mathcal{A}(f \otimes g)$ . Moreover, any  $h \in \mathcal{A}(f \otimes g)$  factorizes as  $h' \circ (f' \otimes g')$  with  $f' \in \mathcal{A}(f), g' \in \mathcal{A}(g)$  and  $h' \in \mathcal{A}(\text{id}_{B \otimes D})$ .

Let  $R: \mathbf{C} \rightarrow \mathbf{D}$  be a lax symmetric monoidal functor and  $F: \mathbf{D} \rightarrow \mathbf{C}$  be a strong monoidal functor. We define resources relative to  $R \circ F = RF$  as pairs  $(A, r)$ , where  $A$  is an object of  $\mathbf{D}$  and  $r$  is an element of the set  $RF(A)$ . Given two resources  $(A, r)$  and  $(B, s)$  we say that a morphism  $f: A \rightarrow X$  is an  *$\mathcal{A}$ -secure morphism*  $(A, r) \rightarrow (B, s)$  if for any  $a \in \mathcal{A}(F(f))$  with  $\text{dom}(a) = F(A)$  there is a  $b \in \mathcal{A}(\text{id}_{F(B)})$  with  $\text{dom}(b) = F(B)$  such that  $R(a)r = R(b)s$ , i.e., for any such  $a$  there is a  $b$  making the following square commutes.

$$\begin{array}{ccc} I_{\text{Set}} = \{*\} & \xrightarrow{s} & RF(B) \\ r \downarrow & & \downarrow R(b) \\ RF(A) & \xrightarrow{R(a)} & R(Y) \end{array}$$

Finally, we say that the attack model  $\mathcal{A}$  is *external* to  $F$  if for every  $f: A \rightarrow B$  in  $\mathbf{D}$ , every attack on  $F(f)$  factorizes as  $a \circ F(f)$  for some  $a \in \mathcal{A}(\text{id}_{F(B)})$ .

**Theorem A.2.** Let  $R: \mathbf{C} \rightarrow \mathbf{D}$  be a lax symmetric monoidal functor,  $F: \mathbf{D} \rightarrow \mathbf{C}$  be a strong monoidal functor, and  $\mathcal{A}$  be an attack model on  $\mathbf{C}$ . Then resources and secure resource conversions form a symmetric monoidal category, with the SMC structure inherited from that of  $\mathbf{D}$ .

*Proof.* The proof is essentially that of [26, Theorem 4.6]. However, as our assumptions are slightly different, some

modifications are necessary, so we give the full proof for the sake of completeness.

We first describe the SMC structure inherited from  $\mathbf{D}$ . The sequential composite of  $f: (A, r) \rightarrow (B, s)$  and  $g: (B, s) \rightarrow (C, t)$  is defined as  $g \circ f: (A, r) \rightarrow (C, t)$ , i.e., by forming the composite in  $\mathbf{D}$ . Similarly, the parallel composite of  $f: (A, r) \rightarrow (B, s)$  and  $g: (C, t) \rightarrow (D, u)$  is given by  $f \otimes g: (A \otimes C, r \otimes t) \rightarrow (B \otimes D, s \otimes u)$ , where  $r \otimes t$  denotes the image of  $(r, t)$  under the map  $RF(A) \times RF(C) \rightarrow RF(A \otimes C)$  given by the lax monoidal structure on  $R \circ F$  and  $s \otimes u$  is defined similarly. Similarly, the coherence isomorphisms (and in particular the symmetries) are given by those in  $\mathbf{D}$ . What remains to show is that this is well defined in that

- Identities, and more generally all coherence isomorphisms, are  $\mathcal{A}$ -secure;
- $\mathcal{A}$ -security is closed under sequential and parallel composition.

The required equations (e.g., associativity or equations between coherence isomorphisms) then follow since they hold in  $\mathbf{D}$ .

We first prove the claim when  $F = \text{id}_{\mathbf{C}}$ . For coherence isomorphisms we prove a stronger claim and show that all isomorphisms are  $\mathcal{A}$ -secure. Let  $f: A \rightarrow B$  be an isomorphism in  $\mathbf{C}$ , and let  $r \in R(A)$ . We claim that  $f$  defines an  $\mathcal{A}$ -secure morphism  $(A, r) \rightarrow (B, R(f)r)$ . Indeed, consider  $a \in \mathcal{A}(f)$  with  $\text{dom}(a) = A$ . Then  $R(a)r = R(a)R(f^{-1})R(f)r$ , so it suffices to show that  $af^{-1} \in \mathcal{A}(\text{id}_B)$ . Property (i) of  $\mathcal{A}$  implies that  $f^{-1} \in \mathcal{A}(f^{-1})$  so that property (ii) gives us  $af^{-1} \in \mathcal{A}(f f^{-1}) = \mathcal{A}(\text{id}_B)$ , as desired.

That  $\mathcal{A}$ -secure maps are closed under  $\circ$  is proved verbatim as in [26, Theorem 4.6]. Assume now that  $f: (A, r) \rightarrow (B, s)$  and  $g: (B, s) \rightarrow (C, t)$  are  $\mathcal{A}$ -secure. Given  $h \in \mathcal{A}(g \circ f)$  with domain  $A$ , factorize it as  $g' \circ f'$  as guaranteed by (ii). As  $f$  is  $\mathcal{A}$ -secure, there is some  $b \in \mathcal{A}(\text{id}_B)$  with  $R(f')r = R(b)s$  and by (ii)  $g'b \in \mathcal{A}(g)$ , so that security of  $g$  implies the existence of  $c \in \mathcal{A}(\text{id}_C)$  such that  $R(g'b)(s) = R(c)t$ . Thus  $R(g'f')r = R(g')R(b)s = R(c)t$  showing that  $g \circ f$  is  $\mathcal{A}$ -secure.

That  $\mathcal{A}$ -secure maps are closed under  $\otimes$  is proved verbatim as in [26, Theorem 4.6]. Let  $f: (A, r) \rightarrow (B, s)$  and  $g: (C, t) \rightarrow (D, u)$  be  $\mathcal{A}$ -secure. Given  $h \in \mathcal{A}(f \otimes g)$  with domain  $A \otimes C$ , factorize it as  $h' \circ (f' \otimes g')$  as guaranteed by (iii). Then security of  $f$  and  $g$  gives us  $b \in \mathcal{A}(\text{id}_B)$  and  $d \in \mathcal{A}(\text{id}_D)$  so that  $R(f')r = R(b)s$  and  $R(g')t = R(d)u$ . We now claim that the following diagram commutes.

$$\begin{array}{ccccc} I_{\text{Set}} = \{*\} & \xrightarrow{(r, t)} & R(A) \times R(C) & \longrightarrow & R(A \otimes C) \\ (s, u) \downarrow & & \downarrow R(f') \times R(g') & & \downarrow R(f' \otimes g') \\ R(B) \times R(D) & \xrightarrow{R(b) \times R(d)} & R(Y) \times R(Z) & \longrightarrow & R(Y \otimes Z) \\ & \searrow & \nearrow & & \downarrow R(h') \\ & & R(B \otimes D) & \xrightarrow{R(b \otimes d)} & R(X) \end{array}$$

Here the unlabeled arrows come from the natural transformation  $R(-) \times R(-) \rightarrow R(- \otimes -)$ , making  $R$  lax monoidal. The top left square commutes by security of  $f$  and  $g$ , and the right-most shape commutes by the factorization  $h = h' \circ (f' \otimes g')$ .

The remaining two sub-diagrams are naturality squares and hence commute. Hence the whole diagram commutes. As the top path equals  $R(h)(r \otimes t)$  and the bottom path equals  $R(h' \circ (b \otimes d))(s \otimes u)$ , we see that  $h' \circ (b \otimes d) \in \mathcal{A}(\text{id}_B \otimes \text{id}_D)$  witnesses that  $f \otimes g$  is  $\mathcal{A}$ -secure.

To prove the claim for an arbitrary strong monoidal  $F$ , observe first that  $f: (A, r) \rightarrow (B, s)$  is  $\mathcal{A}$ -secure if, and only if,  $F(f): (F(A), r) \rightarrow (F(B), s)$  is  $\mathcal{A}$ -secure. As a result, all isomorphisms in  $\mathbf{D}$  are  $\mathcal{A}$ -secure as their images in  $\mathbf{C}$  are, and similarly  $\mathcal{A}$ -secure maps are closed under sequential composition. As  $F$  is strong monoidal, the square

$$\begin{array}{ccc} F(A \otimes C) & \xrightarrow{F(f \otimes g)} & F(B \otimes D) \\ \cong \downarrow & & \uparrow \cong \\ F(A) \otimes F(C) & \xrightarrow{F(f) \otimes F(g)} & F(B) \otimes F(D) \end{array}$$

commutes in  $\mathbf{C}$ . If  $f: (A, r) \rightarrow (B, s)$  and  $g: (C, t) \rightarrow (D, u)$  are  $\mathcal{A}$ -secure, then  $F(f)$  and  $F(g)$  are  $\mathcal{A}$ -secure in. The case  $F = \text{id}_{\mathbf{C}}$  implies that  $F(f) \otimes F(g)$  is  $\mathcal{A}$ -secure so that  $F(f \otimes g)$  is  $\mathcal{A}$ -secure as a composite of secure maps, which means that  $f \otimes g$  is  $\mathcal{A}$ -secure as desired.  $\square$

The following lemma follows immediately from the definition of an attack model being external to a functor.

**Lemma A.3.** *Let  $R: \mathbf{C} \rightarrow \mathbf{D}$  be a lax symmetric monoidal functor,  $F: \mathbf{D} \rightarrow \mathbf{C}$  be a strong monoidal functor, and  $\mathcal{A}$  be an attack model on  $\mathbf{C}$ . Assume that  $\mathcal{A}$  is external to  $F$ . Then for any  $r \in RF(A)$ , any morphism  $f: A \rightarrow B$  defines a secure morphism  $f: (A, r) \rightarrow (B, RF(f)r)$ .*

We now describe how our model theory in Section III fits this general pattern: We set  $F: \mathbf{D} \rightarrow \mathbf{C}$  to be the inclusion mapping  $\mathbf{D}_{\text{prot}} \rightarrow \mathbf{D}_{\text{adv}}$  and we set

$$R = \text{hom}(I, -)/\approx: \mathbf{D}_{\text{adv}} \rightarrow \mathbf{Set}$$

The attack model on  $\mathbf{D}_{\text{adv}}$  is defined by setting first for each representative  $(Z, f, g)$  of a morphism  $X_h \rightarrow X_a \otimes Z, g: Z \otimes Y_1 \rightarrow Y_2)$

$$\mathcal{A}(Z, f, g) := \{[Z, f, g'] \in \mathbf{D}_{\text{adv}} \mid g': Z \otimes Y_1 \rightarrow Y_3\}$$

and then defining

$$\mathcal{A}([Z, f, g]) = \bigcup_{(Z', f', g') \in [Z, f, g]} \mathcal{A}(Z', f', g') .$$

We can then derive Theorem III.9 from Theorem A.2 as follows.

*Alternative proof of Theorem III.9.* We assume that  $\text{id}$  defines an  $\mathcal{A}$ -secure morphism  $r \rightarrow s$ . By construction, the attack model  $\mathcal{A}$  is external to  $\mathbf{D}_{\text{prot}} \rightarrow \mathbf{D}_{\text{adv}}$ , so that by Lemma A.3, the morphism  $[Z, f, \text{id}_{Z \otimes Y_a}]$  defines an  $\mathcal{A}$ -secure morphism  $s \rightarrow [Z, f, \text{id}_{Z \otimes Y_a}]s$ , so the claim follows from  $\mathcal{A}$ -secure maps being closed under  $\circ$ , a consequence of Theorem A.2.  $\square$